

QuantumShield — Risk Scoring Formula Draft (Pillar 2 Spec)

Status: Paper design only — implementation belongs to Week 3 (`scoring/qri_engine.py`).

Date: 2026-09-20

Primary references:

- Ishchukova et al. — Quantum Risk Index (QRI) for cryptographic CVEs (CVSS × Quantum Factor × HNDL exposure)
 - Related prioritisation work (Zelenovic et al. QER-style approaches) from Week 1 literature
-

1. Goal for QuantumShield v1 Scoring

Produce a simple, explainable **Algorithm Risk Score** for every `cryptographic-asset` that the discovery scanner emits.

We do **not** yet have full CVSS base scores or organisational HNDL context, so the first version is deliberately algorithm-centric and confidence-aware.

2. Simplified Formula (v1)

```
Algorithm_Risk_Score = Quantum_Factor × Confidence_Level
```

Where: - `Quantum_Factor` ∈ {1.0, 1.2, 1.5} (see table below) -

`Confidence_Level` ∈ [0.0, 1.0] (taken from the CBOM evidence / scanner confidence)

Optional future extension (once CVSS and HNDL data exist):

```
QRI = CVSS_base × Quantum_Factor × (1 + HNDL_Score2 / 10)
```

(This matches the shape published by Ishchukova et al.)

3. Quantum Factor Assignment

Tier	Quantum_Factor	Algorithms / Families	Justification
Shor-vulnerable	1.5	RSA, ECDSA, DSA, DH, ECDH, Ed25519, classic PKI signatures	Shor's algorithm completely breaks the discrete-log / factoring assumptions. Highest priority for migration. Matches the 1.5 factor used in the empirical QRI study on NVD cryptographic CVEs.
Grover-affected	1.2	AES (esp. ≤ 128 -bit), MD5, SHA-1, older hashes, HMAC with weak hashes	Grover's algorithm provides a quadratic speedup, effectively halving the security level. Still urgent but secondary to asymmetric breaks.
Quantum-safe / stronger	1.0	ML-KEM, ML-DSA, SLH-DSA, AES-256 (practically), SHA-3, SHAKE, properly parameterised PQC	Designed or sufficiently large to resist known quantum attacks at the target security level. Baseline risk.

Notes for implementers: - If key size is known and is classically weak (e.g. RSA-1024), the factor may later be raised further; for v1 keep the family-level factor. - MD5 and SHA-1 receive 1.2 even though they are already classically broken, because the quantum factor still indicates residual quantum-relevant exposure in legacy code.

4. Confidence Level

Taken directly from the scanner / CBOM evidence:

Detection method	Suggested default Confidence
Exact library API call (future AST)	0.95

Detection method	Suggested default Confidence
Strong regex + surrounding context	0.85
Simple string / keyword match	0.70
Heuristic or low-signal match	0.50

`cbom_writer.py` should already emit a confidence value; the scoring engine just multiplies.

5. HNDL Exposure (future)

When organisational context becomes available, add an HNDL score (0–N) based on: - Long-term data retention - Network-exposed channels - High-value sector - Key-material exposure

Then apply the quadratic term from Ishchukova:

```
multiplier = 1 + (HNDL_Score ** 2) / 10
```

For the current PoC this term is omitted (effectively = 1).

6. Output Contract for `qri_engine.py`

Given a CBOM component of type `cryptographic-asset`, the engine shall return:

```
{
  "bom-ref": "...",
  "name": "RSA-2048",
  "quantum_factor": 1.5,
  "confidence": 0.85,
  "algorithm_risk_score": 1.275,
  "tier": "shor-vulnerable",
  "rationale": "Shor-vulnerable asymmetric algorithm"
}
```

Scores can later be normalised or mapped to a 0–10 scale for the dashboard; the raw product is sufficient for prioritisation.

7. Justification Summary

- **Shor vs Grover split** is the dominant scientific consensus and is directly reflected in the empirical QRI work (Ishchukova et al.).
- Keeping the formula multiplicative and transparent allows security teams to understand *why* an algorithm ranks high.
- Starting with algorithm + confidence only lets us ship scoring in Week 3 without waiting for CVSS enrichment or organisational HNDL questionnaires.
- The design is intentionally compatible with the full QRI formula so that future data sources can be plugged in without changing the core engine interface.

This document is the specification that `scoring/qri_engine.py` will be coded against.